

Project Workflow & Action Plan

AI-Based Compiler Optimization Recommendation System

Companion to `minic-language-spec.md`, which has the full grammar and type rules. This document covers the system architecture, team roles, and what to actually do.

What this project is

Compilers normally run a fixed set of optimizations on every program. This project builds a small C-like compiler (MiniC) with a library of toggleable classical optimizations, then trains a model that looks at a new program's static features and predicts which combination will actually help — replacing brute-force search with a fast, learned recommendation.

How it works — two modes

Offline: building the training dataset

1. Benchmark corpus (30-40 MiniC programs)
2. Parse + extract features (P1: AST → IR → feature vector)
3. Try all 64 pass combinations — for each: apply passes, codegen to C, compile with `gcc -O0`, run 15-20 times, take median (P2 + P3)
4. Result: training dataset of (features, combo, timing) rows — roughly 1,900-2,500 total

Online: recommending for new code (the actual product)

1. New MiniC program arrives
2. Parse + extract features — same P1 pipeline, no timing needed here
3. Trained model predicts the best combo directly — no exhaustive search (P4)
4. Apply that combo, codegen, compile, run — report measured speedup vs. an unoptimized baseline

For evaluation, also run the true best combo (from exhaustive search on that same program) behind the scenes, so you can report how close the model's prediction landed to optimal — not just that it beat doing nothing.

Team roles

Role	Owns
P1	Lexer, parser, AST, semantic checks, IR generation, static feature extractor
P2	Optimization pass library: constant folding, dead code elimination, CSE, loop-invariant code motion, strength reduction — each independently toggleable
P3	IR→C codegen (including the wrapper-struct trick from the language spec), benchmark corpus, automated <code>gcc -O0</code> compile-and-time harness
P4	Dataset assembly, ML model training, recommendation demo interface

Frozen decisions

- **Language:** MiniC — see `minic-language-spec.md` for the full grammar and type rules
- **Passes:** constant folding, DCE, CSE, LICM, strength reduction (5 passes → 64 combos)
- **Measurement:** real wall-clock timing, compiled with `gcc -O0` (never `-O2` +, or gcc's own

optimizations wash out your measurements), 15-20 runs per binary, median taken

- **Implementation language for the compiler:**
recommended Python (hand-written recursive-descent parser; fastest to iterate with Claude Code) — confirm with your professor if a specific language is required
- **IR:** three-address code, reusing the format already covered in BCSE307L coursework

Build order & dependencies

Order	Who	Can start when	Notes
1	P1	Immediately	Parser + IR is the critical path — nothing downstream has real data until this exists
2	P2	Design now; run once P1 delivers IR	Verify each pass individually on hand-

Order	Who	Can start when	Notes
			traced IR first
3	P3 (codegen + harness)	Immediately, against mock/hand-written IR	Doesn't need a real front-end to build and test
4	P3 (benchmark corpus)	Immediately	No dependency on anyone
5	P1 (feature extractor)	Alongside the parser	Reads the same AST/IR the parser produces
6	P4	Blocked until 1-5 produce real data	Build against synthetic data now so there's no idle time

Milestones

1. Canonical example compiles, type-checks, produces IR (P1)
2. Each of the 5 passes verified individually on hand-traced IR (P2)
3. Single combo, full pipeline: IR → codegen → `gcc -O0` → binary → runs and outputs exactly 83 for the canonical example — first correctness gate
4. All 64 combos run automatically on one benchmark program, timings logged (P3)
5. Full corpus swept across all 64 combos → real dataset exists (unblocks P4)
6. Model trained, recommends a combo for a held-out program, measured speedup reported (P4)

Start this week, by role

- **P1:** tokenize MiniC's keywords/literals; get a lexer running against the canonical example
- **P2:** hand-trace all 5 passes against the canonical example's TAC on paper before writing code
- **P3:** write ~10 benchmark programs (cover 2D arrays, structs, strings); stub the `gcc -O0` compile-and-time harness against a hand-written C file

- **P4:** decide classification vs. regression framing;
build the training script against fake data

Team-wide sync needed

Agree together on the exact TAC instruction set (opcodes) before P2 and P3 build much further — it's the shared contract both of them depend on, and the most common way this kind of project stalls is the two of them quietly assuming different IR shapes.